#5

CS 0447
Introduction to
Computer Programming

Arrays

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

Arrays and Addresses

Strings, arrays, objects…

- When we wanted to store 4-byte values…
  - We split them up across consecutive bytes
- What about a string?
  - How is a string *represented?*
  - How many bytes is a string?
    - Might be thousands or millions of characters
- *Any* array might be that big too
- The solution to *storing* it in memory is the same
  - but **how do you *access* these big things**
    - they don't fit in registers!
      - – sssssssoooooooooooooooooooooooo…

Addr	Val
0	00
1	30
2	04
3	00
4	DE
5	C0
6	EF
7	BE
8	6C
9	34
A	00
B	01
C	02

What's an array?

- If we did this in C or Java:

```
byte[] arr = {1, 2, 3, 4, 5, ...};
```

- In memory it might look like this
- What memory address is **`arr[0]`** at?
  - What about **`arr[1]`**?
  - What about **`arr[2]`**?
  - What about **`arr[3]`**?
- If an array starts at memory address **A**...
  - ...then item n is at address?
  - **A + n**
    - **Not really**
    - Remember what we talked about last class?

Addr	Val
...	...
F405	06
F404	05
F403	04
F402	03
F401	02
F400	01

What if the array contains ints?

- If we did this:

```
int[] arr = {1, 2, 3};
```

- In memory it'd look like *this*
  - Why are there all these 0s?
  - What endianness is being used here?
    - Is the End-byte(LSB)
      in the Little or in the Big memory address?
- What memory address is `arr[1]` at? `arr[2]`?
- If an array starts at memory address **A**...
  - **...and each item is *b* bytes long...**
  - ...then item *n* is at address **A + (*n* × *b*)**
    - On the last slide, *b* happened to be 1
      - Tricky tricky

Addr	Val
F40B	00
F40A	00
F409	00
F408	03
F407	00
F406	00
F405	00
F404	02
F403	00
F402	00
F401	00
F400	01

Accessing arrays

- If you wanna **print all the values in an array,** how do??

```
for(int i = 0; i < length; i++)
    print(data[i]);
```

- Let's focus on ^this bit^ for now
- **`Data`** is an array of **words,** so how big is each item?
- In this calculation, what is **A**? ***b***? ***n***?
- So what's the address calculation?
  - *Address of item i* = **`data + (i * 4)`**
    - Do you think you could convert that into assembly?
      - Well we haven't done the loop yet…
        - » But we'll get to that

Accessing arrays in MIPS

Making arrays in MIPS

- First you need to make space for it just like a variable
  - How did we write that variable?

```
x: .word 4 # static int x=4;
```

- For a small array you can list all the values:

```
little_array: .word 1, 2, 3, 4
```

- But for a big array, that would be annoying
- So you can write:

```
big_array:    .word 0xBEEFC0DE:100
```

- This fills the array with 100 copies of 0xBEEFC0DE
- Notice how similar these look to variables
  - (that's cause there's not really any difference)

MIPS ISA: putting a variable's address in a register

- If the address calculation needs the address of the array…
  - We've gotta get that address into a register right?
    - Can't add something unless it's in registers

- Remember what the **la** instruction does?

```
la t0, little_array
```

- **la** means **load address**
  - ***it doesn't load anything from memory.***
  - Only lw/lh/lhu/lb/lbu load from memory
  - All the other "loads" (li, la) just "put a value in a register"
- What it does: **`t0`** now contains **little_array**'s ***address***

Accessing arrays in MIPS (let's do it together)

- We want to **print out** the value in **`my_array[3]`**.

```
my_array: .word 1, 2, 3, 4
```

- **What's the address calculation?**
- Now turn that into MIPS
  - Let's come up with the **instructions *first***
    - And *then* decide **which registers to use**
  - How do we put the address of **`my_array`** in a register?
  - Now to translate the math
  - Now we have the address; how do we get the value?
  - How do we print it out?
- If we want to *store* a value into the array…
  - We just use a store instruction instead of a load

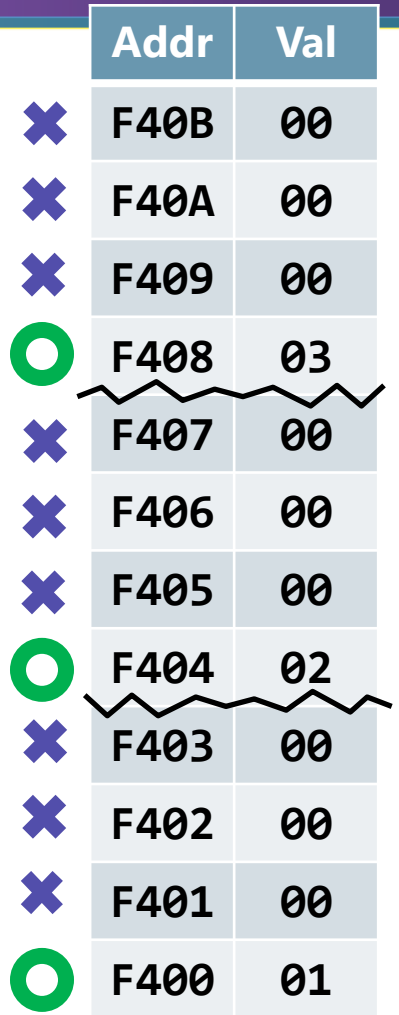
Check code example array_ex1.asm

How does the CPU know that t0 holds an address?

- ***WHAT DO YOU THINK***
  - ***IT DOESN'T***
- **Addresses are just numbers too!!**
  - Which is good! It means we can **do math** on addresses
    - As we just did
    - That's how arrays and strings and stuff *work*
- You can also **have a variable whose value *is an address***
  - Hey
    - 449 students
      - What are these called?
        - » **Pointers** (& is like **la**, * is like **lw/sw**)

Watch out

- Let's remove the **mul** instruction
  - "*fetch address not aligned on word boundary*"?
- In MIPS, all memory accesses must be **aligned**
- **Alignment** is just:
  - the address of a *n*-byte value must be a **multiple of *n***
  - so for 4-byte words…
- that's it, that's all, there's nothing more to it
  - it's not scary



What about strings?

- Strings are also arrays:

```
string: .asciiz “Test”
```

  - The z means it has a zero terminator
  - The same as:

```
string: .byte 84, 101, 115, 116, 0
```

- Be careful with:

```
string: .ascii “Test”
```

  - If you print an unterminated string, strange things happen!
  - The same as:

```
string: .byte 84, 101, 115, 116
```

Check code example arrays_ex3.asm

Questions?

- Is this a valid address for a word?

  0x0800 0001

- How about for a half-word?
- How about for a byte?

- How can I make an array of **halfwords** with 25 elements set to 0?
  - The first **halfword** is in address 0x0800 00F0, where is the second?

- How is a string terminated?